Tutorial I

Desarrollo, estructuración y compilación de una aplicación Android basada en WebView mediante Android Studio

Taller de Innovación y Emprendimiento Tecnológico

Universidad Adolfo Ibáñez

Profesor: Martín Céspedes Escuti

Ayudante Instructor: Artemio Araya Day

17 de octubre de 2026

Índice

Objetivo 3

Detalles de la Aplicación y Herramientas a Utilizar 4

Definición teórica 4

Aplicación de cada lenguaje en la calculadora 6

Bibliotecas y API utilizadas desde Kotlin 7

Bibliotecas de JavaScript 9

Sección de Preguntas 10

Material para desarrollar la Aplicación .apk 20

Descargar e instalar Android Studio 20

Descargar el repositorio de GitHub 23

Abrir el proyecto en Android Studio 23

Configurar el SDK y Kotlin 24

Instalar los componentes del SDK 24

Seleccionar el JDK para Gradle 25

Sincronizar el proyecto 25

Generar el APK de la calculadora 26

Encontrar y guardar el archivo correcto 27

Objetivo

Este tutorial explica cómo preparar y compilar una calculadora para Android utilizando Kotlin, JavaScript, HTML y CSS.

El proceso comienza con la instalación de Android Studio en Windows y la descarga de los archivos del proyecto desde GitHub, continúa con la configuración de las herramientas de compilación y termina con la instalación de la aplicación en el teléfono.

La calculadora utiliza HTML para su estructura, CSS para su presentación, JavaScript para las operaciones y Kotlin para la parte nativa de Android.

Detalles de la Aplicación y Herramientas a Utilizar

Definición teórica

- HTML: Definir la estructura y el contenido: qué elementos existen en la pantalla (MDN Web Docs, s. f.).

- CSS: Definir la presentación visual: cómo se ven y distribuyen los elementos del HTML (MDN Web Docs, s. f.).

- JavaScript: Implementar el comportamiento y la lógica: qué sucede cuando el usuario interactúa y cómo se procesan los datos (MDN Web Docs, s. f.).

- Kotlin: Implementar la integración nativa con Android: comunicación con el sistema operativo, gestión de pantallas y acceso a funciones del dispositivo cuando la aplicación las necesita (Android Developers, s. f.-e).

La calculadora utiliza, desde su código Kotlin, las siguientes clases del SDK (Software Development Kit) de Android, cuyo código fuente está escrito en Java. La parte nativa que hemos desarrollado está escrita en Kotlin y no contiene archivos fuente Java propios (artemioadaysolvers, s. f.).

- Activity: Permite crear la pantalla de la aplicación y gestionar su apertura, pausa y cierre (Android Developers, s. f.-e).

- WebView: Permite mostrar y ejecutar la calculadora desarrollada con HTML, CSS y JavaScript dentro de la aplicación Android (Android Developers, s. f.-a).

- FrameLayout: Actúa como contenedor del WebView dentro de la pantalla de la aplicación (Android Developers, s. f.-e).

- Color: Permite definir el color de fondo del contenedor nativo y del WebView (Android Developers, s. f.-e).

- Uri: Permite examinar las direcciones de los archivos locales que intenta cargar el WebView (Android Developers, s. f.-e).

Estas clases forman parte de Android y se utilizan desde nuestro código Kotlin. No hemos escrito sus archivos Java: son componentes proporcionados por la plataforma. El proyecto de la calculadora no contiene archivos fuente Java propios.

Aplicación de cada lenguaje en la calculadora

- HTML: Definir la estructura de la calculadora, incluyendo los botones de números, los operadores y la pantalla de resultados.

- CSS: Definir la apariencia de la calculadora, incluyendo los colores, los tamaños, la distribución de los botones y la adaptación a diferentes tamaños de pantalla.

- JavaScript: Implementar las operaciones matemáticas, responder a los botones y validar situaciones como la división entre cero.

- Kotlin: Implementar la parte nativa de Android: crear la ventana de la aplicación, mostrar la calculadora mediante un WebView y controlar su ciclo de vida al abrirse, pausarse o cerrarse.

Un **WebView** es un componente de Android que muestra contenido web dentro de una aplicación. En este proyecto muestra los archivos HTML, CSS y JavaScript incluidos en la propia calculadora. No necesita abrir un navegador externo ni descargar una página de Internet. (Android Developers, s. f.-a)

Bibliotecas y API utilizadas desde Kotlin

La aplicación utiliza la biblioteca estándar de Kotlin y las API del Software Development Kit de Android. Una API es un conjunto de funciones y clases que permite utilizar las capacidades de una plataforma. (Android Developers, s. f.-e; JetBrains, s. f.-d)

- kotlin-stdlib: Biblioteca estándar de Kotlin. Reúne herramientas de uso general para trabajar con textos, números, colecciones de datos y errores. Facilita las tareas habituales de programación y ayuda a escribir el código de forma más sencilla y organizada (JetBrains, s. f.-d).

- android.app: Permite organizar las pantallas de una aplicación y gestionar su ciclo de vida. Esto incluye responder cuando una pantalla se abre, pasa a segundo plano, vuelve a mostrarse o se cierra (Android Developers, s. f.-e).

- android.webkit: Permite integrar contenido web dentro de una aplicación Android. Se encarga de mostrar HTML, aplicar CSS y ejecutar JavaScript, además de controlar qué contenido se carga y cómo se presenta (Android Developers, s. f.-a).

- android.view: Proporciona la base para gestionar los elementos visuales de una interfaz. Permite controlar su tamaño, posición, visibilidad y respuesta a la interacción del usuario, así como adaptar el contenido al espacio disponible en la pantalla (Android Developers, s. f.-e).

- android.widget: Ofrece componentes de interfaz preparados para utilizarse, como botones, campos de texto y contenedores. Los contenedores permiten agrupar y distribuir otros elementos dentro de una pantalla (Android Developers, s. f.-e).

- android.os: Permite trabajar con servicios e información del sistema operativo. Entre sus usos están consultar la versión de Android, intercambiar datos entre componentes y manejar información sobre el estado de una pantalla (Android Developers, s. f.-e).

- android.graphics: Se utiliza desde Kotlin para establecer el color de fon-

do del contenedor Android donde se muestra la calculadora. El diseño de los botones, los textos y la pantalla de resultados se define mediante HTML y CSS (Android Developers, s. f.-e).

- android.net: Permite examinar las direcciones de los recursos que intenta cargar el WebView. En esta calculadora ayuda a comprobar que se accede únicamente a los archivos locales incluidos en la aplicación (Android Developers, s. f.-e).

Estas API de Android están incluidas en el SDK (Software Development Kit); no es necesario descargarlas una por una como bibliotecas adicionales (Android Developers, s. f.-e).

Bibliotecas de JavaScript

La calculadora no utiliza bibliotecas externas de JavaScript. Las operaciones están implementadas en engine.js y la interacción con los botones está en app.js, utilizando JavaScript y las funciones disponibles en el WebView. No se utilizan React, React Native ni jQuery (artemioadaysolvers, s. f.).

La librería engine.js contiene la lógica de cálculo. Guarda los números y la operación seleccionada, realiza los cálculos y detecta errores como dividir entre cero. La app.js conecta los botones y la pantalla con esa lógica. Detecta las pulsaciones, envía las instrucciones a engine.js y muestra el resultado (artemioadaysolvers, s. f.).

Sección de Preguntas

¿Por qué utilizar Kotlin y no Java o C++ directamente si Kotlin llama a funcionalidades por medio de la API SDK (Software Development Kit) que a su vez utilizan esos lenguajes?

El Kotlin convencional (Kotlin/JVM) no genera directamente código máquina durante la compilación. Primero genera bytecode, un lenguaje intermedio que puede ejecutar la máquina virtual de Java (JVM). (JetBrains, s. f.-c)

Java y Kotlin se compilan al mismo tipo de bytecode, que posteriormente es interpretado o traducido a código máquina por la máquina virtual de Java (JVM), permitiendo que el procesador ejecute las instrucciones del programa. (JetBrains, s. f.-c; Oracle, 2025)

Kotlin fue diseñado, entre otras cosas, para reducir la cantidad de código necesario respecto de Java, manteniendo la compatibilidad con su ecosistema. (JetBrains, s. f.-c)

Kotlin puede llamar directamente a funciones y clases escritas en Java, sin necesidad de convertirlas a Kotlin. Esto se conoce como interoperabilidad entre Kotlin y Java, y es posible porque ambos lenguajes pueden utilizar la misma máquina virtual (JVM). (JetBrains, s. f.-a)

JVM ejecuta un programa Kotlin como si hubiese sido compilado desde Java. Esto es posible porque ambos lenguajes pueden generar el mismo tipo de bytecode. Pero hay una diferencia importante: Kotlin no se transforma en código fuente Java. (JetBrains, s. f.-c)

Kotlin es un lenguaje de programación independiente, desarrollado por JetBrains. No está escrito en Java, aunque tiene una estrecha relación con ese lenguaje. (JetBrains, s. f.-c)

Kotlin fue diseñado para aprovechar el ecosistema de Java, especialmente su máquina virtual (JVM) y sus bibliotecas. Ambos tienen compiladores distintos, pero pueden generar el mismo tipo de bytecode. (JetBrains, s. f.-c)

La JVM (Java Virtual Machine), o Máquina Virtual de Java, es un programa que permite ejecutar aplicaciones compiladas en bytecode, como las desarrolladas en Java o Kotlin. Su función principal es transformar o interpretar las instrucciones del bytecode para que el procesador (CPU) pueda ejecutarlas. (Oracle, 2025)

La CPU (Central Processing Unit), o Unidad Central de Procesamiento, es el procesador principal de un computador. Es un componente físico (hardware) encargado de ejecutar las instrucciones de los programas. Por ejemplo, los procesadores Intel Core, AMD Ryzen y Apple M son CPU.

Cuando programas en Kotlin o Java, el código debe transformarse en instrucciones que la CPU pueda ejecutar. (Oracle, 2025)

La CPU es el procesador encargado de ejecutar las instrucciones de las aplicaciones y del sistema operativo Android.

¿Por qué utilizar JavaScript y no React para desarrollar la aplicación?

React es una biblioteca de JavaScript que permite desarrollar interfaces de usuario de manera estructurada, modular y eficiente. A diferencia del desarrollo con JavaScript puro, donde gran parte de la manipulación y actualización de los elementos HTML debe realizarse manualmente, React permite organizar la interfaz de una aplicación en componentes, piezas que se definen una vez y pueden emplearse en distintas partes de la pantalla. Por ejemplo, en una calculadora, un mismo componente de botón puede servir para todos los números, cambiando el texto que muestra y la acción que realiza al pulsarlo. Las características comunes de esos botones pueden modificarse desde un único lugar, lo que facilita mantener un diseño uniforme. (React, s. f.)

Facilitando su mantenimiento, escalabilidad y desarrollo. Por esta razón, se recomienda utilizar React para aplicaciones que incorporen múltiples pantallas, autenticación de usuarios, mapas interactivos, conexión con bases de datos y comunicación con servicios externos, como Microsoft Azure.

Además, React puede combinarse con HTML, CSS y JavaScript para integrar la aplicación web en un proyecto de Android Studio y generar un archivo APK. De esta manera, es posible desarrollar una aplicación móvil aprovechando tecnologías web, sin necesidad de implementar toda la interfaz mediante Kotlin o Java. (React, s. f.; Android Developers, s. f.-a)

¿Cuando conviene usar JavaScript sin React.js para desarrollar la aplicación?

JavaScript puro puede ser más conveniente para desarrollar componentes muy personalizados desde cero, pero React.js no limita su creación. Incluso se pueden desarrollar esos componentes con JavaScript puro e integrarlos posteriormente en una aplicación React.js. (React, s. f.)

Además puede preferirse sobre React.js para desarrollar componentes poco convencionales, pero no porque React limite lo que puedes programar, sino porque algunas funcionalidades no necesitan su sistema de componentes. (React, s. f.)

¿Que diferencia hay entre React.js y React Native?

La principal diferencia entre React.js y React Native es que React.js se utiliza principalmente para desarrollar interfaces web, mientras que React Native permite desarrollar aplicaciones móviles utilizando componentes nativos de Android e iOS. (React, s. f.; React Native, s. f.-c)

React Native es un framework de desarrollo de aplicaciones móviles, es decir, un conjunto de bibliotecas, herramientas y componentes de software que permite crear aplicaciones para Android e iOS utilizando JavaScript o TypeScript. (React Native, s. f.-c, s. f.-e)

Es un conjunto de archivos de código y bibliotecas que instalas en tu proyec-

to. Incluye componentes desarrollados en JavaScript, C++, Java/Kotlin y Objective-C/Swift. (React Native, s. f.-b)

React Native proporciona herramientas para conectar tu código JavaScript con las funcionalidades y los componentes nativos del teléfono. (React Native, s. f.-b)

React Native no garantiza al 100 % que una aplicación funcione correctamente en Android e iOS, pero sí está diseñado para desarrollar aplicaciones para ambos sistemas operativos utilizando una gran parte del mismo código. (React Native, s. f.-c)

React Native permite compartir gran parte del código entre Android e iOS, pero tendrás que compilar, probar y posiblemente adaptar la aplicación para cada sistema operativo. (React Native, s. f.-c)

React Native se instala en tu proyecto y después puedes importar sus componentes y funciones, igual que haces con una biblioteca de JavaScript. (React Native, s. f.-c)

La diferencia es que React Native no es una sola biblioteca: es un framework, es decir, un conjunto de bibliotecas, herramientas y código que proporciona una estructura para desarrollar aplicaciones móviles. (React Native, s. f.-c)

Puedes entender React Native como una «superbiblioteca» de JavaScript.

React Native es como una «superbiblioteca» que permite comunicar JavaScript con componentes desarrollados en diferentes lenguajes de programación (React Native, s. f.-b). Principalmente:

- JavaScript / TypeScript: para programar la aplicación.
- Kotlin / Java: para implementar funcionalidades nativas de Android.
- Swift / Objective-C: para implementar funcionalidades nativas de iOS.
- C++: para componentes internos y módulos nativos compartidos. Ambos utilizan JavaScript o TypeScript y comparten conceptos como componentes, estados y propiedades (props). (React, s. f.; React Native, s. f.-c)

React.js y React Native están diseñadas para plataformas diferentes (React, s. f.; React Native, s. f.-c).

React Native está diseñado específicamente para desarrollar aplicaciones móviles y puede ofrecer mejor rendimiento que una aplicación basada en HTML, CSS y JavaScript dentro de un WebView. Sin embargo, no siempre será más rápido ni más eficiente. (React Native, s. f.-a; Android Developers, s. f.-a; Biørn-Hansen et al., 2019)

La diferencia fundamental es que React Native utiliza componentes nativos de Android e iOS, mientras que HTML y CSS necesitan un motor de navegador para representar la interfaz (React Native, s. f.-b; Android Developers, s. f.-a).

React Native puede reducir el procesamiento requerido por la CPU y el consumo energético en determinadas tareas de renderizado, en comparación con tecnologías basadas en WebView. En un estudio experimental que comparó React Native, Ionic y otros frameworks, se observó que React Native presentó el menor consumo de CPU y energía durante la ejecución de animaciones continuas de múltiples imágenes. Sin embargo, esta ventaja depende de la carga de trabajo y no implica necesariamente una menor cantidad de instrucciones de máquina en todas las aplicaciones (Oliveira et al., 2023).

Operación	React Native: CPU	Ionic (WebView): CPU	React Native: energía	Ionic (WebView): energía	Mejor resultado y ventaja	Mejor calidad del resultado
Animación de imágenes	+68 %	+88 %	+130 %	+364 %	**React Native:** menor consumo de CPU y energía.	**No determinado por el estudio**
Desplazamiento de listas	+661 %	+348 %	+975 %	+691 %	**Ionic:** menor consumo de CPU y energía.	**React Native:** mayor fluidez, con menos fotogramas retrasados.
Menú lateral (drawer)	+696 %	+138 %	+871 %	+233 %	**Ionic:** menor consumo de CPU y energía.	**React Native:** apertura y cierre más fluidos.
Cambio entre pantallas	+612 %	+296 %	+186 %	+53 %	**Ionic:** menor consumo de CPU y energía.	**React Native:** transiciones más fluidas.
Inserción en base de datos	+488 %	+323 %	+343 %	+201 %	**Ionic:** menor consumo de CPU y energía.	**No determinado por el estudio**

Nota: Los porcentajes representan el sobrecosto de CPU y energía respecto de una implementación nativa en Java (Oliveira et al., 2023). La columna de eficiencia identifica el menor sobrecosto. La columna sobre calidad se limita a la fluidez visual (fotogramas retrasados) medida para listas, menús y transiciones por Huber et al. (2020). Son experimentos distintos y no deben interpretarse como una única medición.

Fuente: elaboración propia con base en Oliveira et al. (2023) y Huber et al. (2020).

Chromium puede resultar más eficiente que React Native en ciertas tareas porque su motor de renderizado dispone de rutas muy optimizadas para operaciones web. Utilizar componentes nativos no garantiza menor actividad de los circuitos de la CPU ni menor consumo energético (Harrelson, 2021; Oliveira et al., 2023).

- React.js: es una biblioteca de JavaScript utilizada para construir interfaces de usuario, principalmente para páginas y aplicaciones web (React, s. f.).

- React Native: es un framework que utiliza React para desarrollar aplicaciones móviles con interfaces nativas de Android e iOS (React Native, s. f.-c).

Característica	HTML/CSS en WebView	React Native
Interfaz nativa	No, interfaz web	Sí
Animaciones de interfaz	Buen rendimiento si están optimizadas	Puede aprovechar animaciones nativas
Listas extensas	Requiere técnicas de optimización	Incluye componentes como FlatList
Consumo de RAM	Depende del contenido y motor web	Depende de los componentes y runtime
Integración con Android/iOS	Mediante APIs web o puentes nativos	Mediante APIs y módulos nativos
Renderizado 3D con Three.js	Integración directa con tecnologías web	Requiere integración gráfica compatible
Rendimiento general	Puede ser excelente	Puede ser excelente

Fuente: elaboración propia con base en la documentación de Android Developers (s. f.-a) y React Native (s. f.-a, s. f.-b).

¿Que diferencia hay entre WebView y React Native?

La principal diferencia entre WebView y React Native es que WebView permite desarrollar interfaces utilizando HTML, CSS y JavaScript, las cuales se ejecutan dentro de un motor de navegador integrado en la aplicación, mientras

que React Native utiliza JavaScript o TypeScript para construir interfaces mediante componentes nativos de Android e iOS, sin depender de un navegador para renderizarlas. En WebView, se puede utilizar Kotlin para acceder a las funcionalidades nativas del dispositivo y comunicarlas con JavaScript, mientras que React Native proporciona APIs y módulos para acceder a estas funcionalidades. WebView ofrece mayor flexibilidad para reutilizar tecnologías web, mientras que React Native suele ofrecer una experiencia más cercana a una aplicación nativa, especialmente en interfaces móviles complejas.

React Native utiliza componentes nativos y dispone de optimizaciones para el renderizado, las animaciones y la comunicación entre JavaScript y el sistema operativo (React Native, s. f.-a, s. f.-b, s. f.-d).

WebView ejecuta su interfaz dentro de Chromium, un motor web muy optimizado, pero que también requiere memoria y recursos adicionales para funcionar (Android Developers, s. f.-d).

React Native puede presentar una mayor eficiencia que WebView en aplicaciones móviles convencionales, debido a que utiliza componentes nativos de Android e iOS y dispone de optimizaciones para el renderizado, las animaciones y la comunicación entre JavaScript y el sistema operativo. A diferencia de WebView, no necesita ejecutar un motor de navegador completo basado en Chromium para representar su interfaz, evitando los recursos asociados a dicho motor. En este contexto, se entiende por eficiencia la capacidad de una aplicación para ofrecer un rendimiento fluido, tiempos de respuesta reducidos y un uso optimizado de recursos computacionales, como memoria RAM, CPU y energía. Sin embargo, esto no significa que React Native consuma necesariamente menos recursos que WebView, ya que el rendimiento y el consumo dependen de la complejidad, arquitectura y optimización de cada aplicación (React Native, s. f.-a, s. f.-b; Android Developers, s. f.-d).

Característica	HTML/CSS en WebView	React Native
Ventaja: desarrollo de interfaces	Permite utilizar HTML, CSS y JavaScript directamente.	Permite crear interfaces con JavaScript o TypeScript y componentes nativos.
Ventaja: personalización	Gran flexibilidad visual mediante CSS y tecnologías web.	Ofrece componentes nativos y bibliotecas de interfaz.
Ventaja: reutilización	Permite reutilizar interfaces y código de aplicaciones web.	Permite compartir gran parte del código entre Android e iOS.
Ventaja: gráficos interactivos	Integración directa con Canvas, SVG, WebGL y Three.js.	Permite utilizar bibliotecas gráficas compatibles con React Native.
Desventaja: rendimiento	Las interfaces complejas pueden requerir mayor optimización del motor web.	Las interfaces complejas también pueden requerir optimización del renderizado.
Desventaja: acceso nativo	Requiere puentes de comunicación con Kotlin o Java para ciertas funciones.	Algunas funcionalidades requieren módulos nativos adicionales.
Desventaja: experiencia nativa	Los componentes HTML no son controles nativos de Android o iOS.	No utiliza directamente HTML ni CSS tradicional.
Desventaja: mantenimiento	Debe mantenerse la integración entre WebView y el código nativo.	Depende de bibliotecas, módulos y compatibilidad del ecosistema React Native.

Fuente: elaboración propia con base en la documentación de Android Developers (s. f.-a) y React Native (s. f.-a, s. f.-b).

¿Como se relaciona Inonic con WebView?

Ionic es un framework de desarrollo de aplicaciones móviles que permite construir interfaces mediante HTML, CSS y JavaScript. Su relación con WebView radica en que utiliza este componente para ejecutar y renderizar las interfaces web dentro de aplicaciones Android e iOS. Generalmente, Ionic se integra con Capacitor, una herramienta que permite acceder a funcionalidades nativas del dispositivo, como la cámara, los sensores, el almacenamiento y el Bluetooth, mediante plugins o código nativo. A diferencia de una aplicación desarrollada directamente con WebView y Kotlin, Ionic proporciona componentes de interfaz prediseñados y herramientas para facilitar el desarrollo multiplataforma. Sin embargo, ambas arquitecturas pueden utilizar el mismo motor de renderizado web, por lo que sus diferencias de rendimiento dependen de la implementación, las bibliotecas utilizadas y la complejidad de la aplicación. Por esta razón, los resultados experimentales obtenidos con Ionic no necesariamente representan el rendimiento de todas las aplicaciones desarrolladas con WebView (Ionic, s. f.; Capacitor, s. f.).

Material para desarrollar la Aplicación .apk

- Un ordenador Windows de 64 bits que cumpla los requisitos de Android Studio (Android Developers, 2026).
- Conexión a Internet para descargar las herramientas y sincronizar el proyecto por primera vez.
- El proyecto fuente completo de la calculadora, versión 1.1.1.
- Un teléfono con Android 6.0 o posterior y con el componente Android System WebView disponible y actualizado. Android 15 está dentro del intervalo admitido por la configuración del proyecto (Android Developers, s. f.-f).
- Un sistema que permita transferir el .apk (app compilada) hacia el dispositivo Android.

El **APK** es el archivo instalable de la aplicación Android. El ZIP contiene los archivos necesarios para construir ese APK. Para probar la app utilizará el teléfono Android; no es necesario configurar un emulador (Android Developers, s. f.-b).

Descargar e instalar Android Studio

Las instrucciones de instalación y los requisitos del equipo se encuentran en la documentación oficial. (Android Developers, 2026)

1. Abre la página oficial de Android Studio: https://developer.android.com/studio
2. Descarga una versión estable para Windows compatible con Android Gradle Plugin 8.13.2. Elige el instalador con extensión .exe.
3. Ejecuta el instalador y sigue sus indicaciones. Puedes conservar la carpeta de instalación propuesta.
4. Abre Android Studio al terminar. Si pregunta si deseas importar una configuración anterior y es tu primera instalación, elige no importarla.
5. En el asistente inicial, selecciona la instalación **Standard** si está disponible. Si utilizas una configuración personalizada, incluye el Android SDK (Software Development Kit).
6. Revisa las licencias y acéptalas si estás de acuerdo. Espera a que termine la descarga de los componentes y pulsa **Finish**.

El **SDK de Android**, del inglés *Software Development Kit*, reúne las herramientas y las API necesarias para desarrollar aplicaciones Android. **En esta aplicación,**

el SDK de Android se utiliza con Kotlin: nuestro código Kotlin utiliza sus API para crear la pantalla, mostrar la calculadora y gestionar su ciclo de vida. Aunque algunas clases de esas API estén implementadas en Java, pueden utilizarse desde Kotlin sin escribir archivos Java propios (Android Developers, s. f.-e; JetBrains, s. f.-a).

El SDK de Android sirve para ambas cosas: desarrollar y compilar la app para obtener el .apk

- **Para desarrollarla:** proporciona las API que utilizamos desde Kotlin para crear la pantalla, mostrar el WebView y acceder a funciones de Android (Android Developers, s. f.-a, s. f.-e).
- **Para compilarla:** incluye herramientas que Gradle utiliza para preparar los recursos y construir el APK (Android Developers, 2025).

El SDK se instala en el ordenador. Cuando abres la app en el teléfono, esta utiliza los componentes de Android que ya están presentes en el dispositivo (Android Developers, 2026).

Descargar el repositorio de GitHub

Repositorio del taller:

https://github.com/artemioadaysolvers/App-Tutorial-I

1. Abre el enlace anterior en tu navegador. Si el repositorio requiere acceso, inicia sesión con una cuenta autorizada.
2. En la página del repositorio, pulsa **Code**.
3. Selecciona **Download ZIP**. Este procedimiento descarga una copia de los archivos de la rama seleccionada. (GitHub, s. f.)
4. Busca el ZIP en las descargas del ordenador.
5. Haz clic derecho y selecciona **Extraer todo**. Guarda los archivos en una carpeta local fácil de identificar, por ejemplo dentro de C:\Proyectos.

Abrir el proyecto en Android Studio

1. En la pantalla inicial de Android Studio, pulsa **Open**. Si ya tienes otro proyecto abierto, utiliza **File >Open**.
2. Busca la carpeta extraída del proyecto completo.
3. Selecciona **CalculadoraSencillaKotlin**, la carpeta que contiene los archivos de la aplicación.
4. Abre el proyecto y espera a que Android Studio termine de reconocer sus archivos.

Abre el proyecto extraído, no el archivo ZIP. No necesitas crear otro proyecto ni copiar los archivos dentro de una plantilla nueva.

Los archivos de configuración terminan en .gradle.kts porque utilizan Kotlin para configurar la compilación (JetBrains, s. f.-b).

Configurar el SDK y Kotlin

La versión 1.1.1 del proyecto tiene configuradas estas herramientas: (artemioadaysolvers, s. f.)

- Android Gradle Plugin: **8.13.2**.
- Kotlin: **2.3.20**.
- Gradle: **8.13**.
- Android SDK Platform: **36**, correspondiente a Android 16.
- Android SDK Build-Tools: **35.0.0**, versión predeterminada del complemento de Android utilizado.
- JDK: **17**, versión que se utilizará en este tutorial.

Android Gradle Plugin 8.13.2 admite Kotlin 2.3 y requiere, como mínimo, Gradle 8.13 y JDK 17. (Android Developers, 2025)

El soporte de Kotlin ya está configurado en build.gradle.kts y app/build.gradle.kts. Android Studio descargará el complemento y sus dependencias durante la sincronización. No es necesario instalar Kotlin manualmente. (JetBrains, s. f.-b)

Instalar los componentes del SDK

1. Abre **Tools >SDK Manager**. Desde la pantalla de bienvenida también puede estar disponible en **More Actions >SDK Manager**.
2. En **SDK Platforms**, selecciona **Android SDK Platform 36**, correspondiente a Android 16, si todavía no está instalada.
3. En **SDK Tools**, comprueba que estén instaladas las **Android SDK Platform-Tools**.
4. Instala **Android SDK Build-Tools 35.0.0** si no está disponible. Para seleccionar una versión concreta, activa **Show Package Details**.
5. Pulsa **Apply**, revisa las licencias y espera a que termine la instalación.

Instalar la plataforma 36 en el ordenador no exige que el teléfono tenga Android 16. El proyecto permite instalar la aplicación desde Android 6.0 porque su valor minSdk es 23; compileSdk y targetSdk están configurados en 36 (Android Developers, s. f.-f).

Seleccionar el JDK para Gradle

1. Con el proyecto abierto, entra en **File >Settings**.
2. Abre **Build, Execution, Deployment >Build Tools >Gradle**.
3. En **Gradle JDK**, selecciona **JDK 17**. Si no está disponible, utiliza **Download JDK**, elige la versión 17 y completa la descarga.
4. Pulsa **Apply** y después **OK**.

El **JDK**, del inglés *Java Development Kit*, proporciona el entorno que Gradle necesita para ejecutar las herramientas de compilación. Se utiliza también al compilar proyectos escritos en Kotlin. Su presencia no significa que debas escribir una parte de la calculadora en Java. (Android Developers, s. f.-c)

Sincronizar el proyecto

1. Si aparece **Sync Now**, pulsa esa opción. También puedes utilizar **File >Sync Project with Gradle Files**.
2. Mantén Internet conectado mientras Gradle descarga los componentes necesarios.
3. Si aparece **Install missing SDK packages**, revisa e instala los componentes indicados.
4. Espera a que termine la sincronización. Si falla, revisa el mensaje de error antes de continuar.

Gradle organiza la compilación y reúne los recursos de la aplicación. El proyecto incluye Gradle Wrapper, que selecciona y descarga la versión indicada en sus archivos. No hace falta instalar Gradle por separado. Para generar el APK de esta calculadora tampoco se requieren Node.js, npm ni Capacitor (Gradle, s. f.).

Conserva las versiones configuradas del proyecto durante esta primera compilación. Una sugerencia para actualizar herramientas no significa necesariamente que debas cambiarlas para generar el APK.

Generar el APK de la calculadora

1. Abre **View >Tool Windows >Build Variants** y selecciona **debug** para el módulo **app**.
2. En el menú superior, pulsa **Build**.
3. Selecciona **Generate Bundle(s) / APK(s)** y después **Generate APK(s)**. Según la versión de Android Studio, la primera opción también puede aparecer como **Generate App Bundles or APKs** o como **Build Bundle(s) / APK(s) >Build APK(s)**.
4. Espera a que finalice el proceso. La primera compilación puede tardar más porque descarga herramientas y prepara archivos.
5. Comprueba que el resultado indique BUILD SUCCESSFUL.

Para instalar manualmente en el teléfono, elige **APK**. El archivo con extensión .aab, llamado Android App Bundle, se utiliza en otro procedimiento de distribución (Android Developers, s. f.-b).

Para esta práctica, la variante debug ya queda firmada con una clave de desarrollo. No hace falta elegir **Generate Signed App Bundle or APK**; ese asistente permite preparar una versión con una clave de distribución propia. (Android Developers, s. f.-b)

Encontrar y guardar el archivo correcto

Al terminar, pulsa **Locate** en la notificación de Android Studio si aparece. También puedes buscar el resultado desde el Explorador de archivos de Windows, dentro de la carpeta del proyecto:

```
app/build/outputs/apk/debug/app-debug.apk
```

El archivo que debes copiar al teléfono se llama **app-debug.apk**. Puedes guardar una copia en Descargas del ordenador o en otra carpeta que identifiques fácilmente (Android Developers, s. f.-b).

BUILD SUCCESSFUL confirma que la compilación terminó correctamente. Después debes instalar y abrir la aplicación para comprobar su funcionamiento en el teléfono (Gradle, s. f.).

- index.html: estructura de la pantalla y botones.
- styles.css: presentación y tamaños.
- engine.js: operaciones y estado de la calculadora.
- app.js: interacción de los botones y el teclado.

La parte nativa de Android está en:

```
app/src/main/kotlin/com/ejemplo/calculadorasencilla/
    MainActivity.kt
```

El archivo MainActivity.kt crea la ventana, configura el WebView, carga los archivos locales y gestiona las pausas y el cierre de la aplicación. Los cálculos continúan implementados en JavaScript (artemioadaysolvers, s. f.).

Otros archivos útiles son:

- app/src/main/res/values/strings.xml: nombre visible de la aplicación.
- app/src/main/AndroidManifest.xml: configuración principal de Android y declaración de la actividad.
- app/build.gradle.kts: identificador de instalación, versiones y niveles del SDK.

Referencias

Android Developers. (s. f.-a). *Build web apps in WebView*. https://developer.android.com/develop/ui/views/layout/webapps/webview

Android Developers. (s. f.-b). *Build your app for release to users*. https://developer.android.com/build/build-for-release

Android Developers. (s. f.-c). *Java versions in Android builds*. https://developer.android.com/build/jdks

Android Developers. (s. f.-d). *Manage and diagnose WebView memory*. https://developer.android.com/develop/ui/views/layout/webapps/manage-webview-memory

Android Developers. (s. f.-e). *Package index*. https://developer.android.com/reference/packages

Android Developers. (s. f.-f). *Uses-sdk*. https://developer.android.com/guide/topics/manifest/uses-sdk-element

Android Developers. (2025). *Android Gradle plugin 8.13.0 (September 2025)*. https://developer.android.com/build/releases/agp-8-13-0-release-notes

Android Developers. (2026). *Install Android Studio*. https://developer.android.com/studio/install

artemioadaysolvers. (s. f.). *App-Tutorial-I* [Repositorio de código]. GitHub. https://github.com/artemioadaysolvers/App-Tutorial-I

Biørn-Hansen, A., Grønli, T.-M., & Ghinea, G. (2019). Animations in cross-platform mobile applications: An evaluation of tools, metrics and performance. *Sensors, 19*(9), 2081. https://doi.org/10.3390/s19092081

Capacitor. (s. f.). *Capacitor documentation*. https://capacitorjs.com/docs

GitHub. (s. f.). *Downloading source code archives*. https://docs.github.com/en/repositories/working-with-files/using-files/downloading-source-code-archives

Gradle. (s. f.). *Gradle Wrapper*. https://docs.gradle.org/current/userguide/gradle_wrapper.html

Harrelson, C. (2021). *RenderingNG architecture*. Chrome for Developers. https://developer.chrome.com/docs/chromium/renderingng-architecture

Huber, S., Demetz, L., & Felderer, M. (2020). Analysing the performance of mobile cross-platform development approaches using UI interaction scenarios. En *Software technologies* (pp. 40-57). Springer. https://doi.org/10.1007/978-3-030-52991-8_3

Ionic. (s. f.). *Ionic Framework*. https://ionicframework.com/docs

JetBrains. (s. f.-a). *Calling Java from Kotlin*. https://kotlinlang.org/docs/java-interop.html

JetBrains. (s. f.-b). *Configure a Gradle project*. https://kotlinlang.org/docs/gradle-configure-project.html

JetBrains. (s. f.-c). *FAQ*. https://kotlinlang.org/docs/faq.html

JetBrains. (s. f.-d). *kotlin-stdlib: Core API*. https://kotlinlang.org/api/core/kotlin-stdlib/

MDN Web Docs. (s. f.). *Aprende desarrollo web*. https://developer.mozilla.org/es/docs/Learn_web_development

Oliveira, W., Moraes, B., Castor, F., & Fernandes, J. P. (2023). Analyzing the resource usage overhead of mobile app development frameworks. En *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering* (pp. 152-161). Association for Computing Machinery. https://doi.org/10.1145/3593434.3593487

Oracle. (2025). *The Java Virtual Machine Specification, Java SE 25 edition: Chapter 1. Introduction*. https://docs.oracle.com/javase/specs/jvms/se25/html/jvms-1.html

React. (s. f.). *Describing the UI*. https://react.dev/learn/describing-the-ui

React Native. (s. f.-a). *Animations*. https://reactnative.dev/docs/animations

React Native. (s. f.-b). *Architecture overview*. https://reactnative.dev/architecture/overview

React Native. (s. f.-c). *Core components and APIs*. https://reactnative.dev/docs/components-and-apis

React Native. (s. f.-d). *Fabric*. https://reactnative.dev/architecture/fabric-renderer

React Native. (s. f.-e). *Introduction*. https://reactnative.dev/docs/getti

```
ng-started
```